

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并程序序设计

批改人：

实验	3-Pthreads并行矩阵乘法与数组求和	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.4.16

实验目的

- 掌握Pthreads创建线程、多线程计算与合并线程的API。
- 掌握Pthreads创建与合并线程时的内存管理。
- 继续练习并行矩阵乘法与矩阵分块的方法。

实验过程与核心代码

并行矩阵乘法

```
int main(int argc, char *argv[]){
    // 获取m,n,k
    int m = atoi(argv[1]);
    int n = atoi(argv[2]);
    int k = atoi(argv[3]);
    int threads = atoi(argv[4]);

    // 分配内存
    double *A = (double *)malloc(m * n * sizeof(double));
    double *B = (double *)malloc(n * k * sizeof(double));
    double *C = (double *)malloc(m * k * sizeof(double));
    double *C_mkl = (double *)malloc(m * k * sizeof(double));
    // 初始化
    srand(time(NULL));
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            A[i * n + j] = rand() % 1000;
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < k; j++) {
            B[i * n + j] = rand() % 1000;
        }
    }
}
```

首先，从输入中获取矩阵规模，为输入矩阵、结果矩阵和验证矩阵分配内存，并使用随机数初始化两个输入矩阵。

```
// 创建线程
pthread_t *thread = (pthread_t *)malloc(threads * sizeof(pthread_t));
matrix *mtx = (matrix *)malloc(threads * sizeof(matrix));

clock_t start = clock();
int row_per_thread = m / threads;
for(int i=0; i<threads;i++){
    int start_row = i * row_per_thread;
    int end_row = (i==threads-1)?m:(i+1)*row_per_thread;
    mtx[i].m = m;
    mtx[i].n = n;
    mtx[i].k = k;
    mtx[i].A = A;
    mtx[i].B = B;
    mtx[i].C = C;
    mtx[i].thread = threads;
    pthread_create(&thread[i], NULL, multiply, &mtx[i]);
}

// 等待所有线程结束
for(int i = 0; i < threads; i++) {
    pthread_join(thread[i], NULL);
}
clock_t end = clock();
```

随后，使用 `pthread` API 创建线程、合并线程。

```
typedef struct{
    int m;
    int n;
    int k;
    double *A,*B,*C;
    int id;
    int thread;
}matrix;
```

在多线程计算中，传递的数据是一个结构体类型。结构体包含了矩阵规模、指向矩阵的指针、线程ID、总线程数。

`pthread_create` 函数用于创建线程，传入四个参数。第一个参数是一个指向 `pthread_t` 类型的指针，用于保存新创建的线程的线程标识符。系统会为每个线程分配一个唯一的ID。第二个参数是一个指向线程属性对象的指针，这里使用默认属性，所以是NULL。第三个参数是一个指向新线程开始执行的函数的指针。第四个参数是一个指向传递给线程的参数的指针。

`pthread_join` 函数用于等待线程结束，传入两个参数。第一个参数是要等待的线程的标识符，第二个参数用于接受线程的返回值。

```

void *multiply(void *args){
    matrix *params = (matrix *)args;
    int m = params->m;
    int n = params->n;
    int k = params->k;
    double *A = params->A;
    double *B = params->B;
    double *C = params->C;
    int id = params->id;
    int thread = params->thread;

    //获得计算起止位置
    int row_per_thread = m / thread;
    int start_row = id * row_per_thread;
    int end_row = (id==thread-1)?m:(id+1)*row_per_thread;

    //分块计算
    for (int i = start_row; i < end_row; i++) {
        for (int j = 0; j < k; j++) {
            C[i * k + j] = 0;
            for (int l = 0; l < n; l++) {
                C[i * k + j] += A[i * n + l] * B[l * k + j];
            }
        }
    }
    pthread_exit(NULL);
}

```

每个线程开始执行的函数如图。首先计算这个线程需要计算的矩阵的起始行和结束行，再分块计算矩阵乘法。需要注意的是，在计算结束行时，需要考虑最后一个线程的情况。最后一个线程需要处理所有剩下的行。

```

//验证计算
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, m, k, n, 1.0, A, n, B, k, 0.0, C_mkl, k);
verify(&C, &C_mkl, m, k);

```

对计算结果进行验证。使用MKL矩阵乘法进行验算，并逐一比较。如果两个矩阵在所有位置上的值都相同，就输出“Results Correct”，否则输出不一致的位置和两次计算的值。

并行数组求和

```

typedef struct{
    int *arr;
    int *part_sum;
    int start;
    int end;
}thread_arr;

```

并行数组求和时给线程传入的数据结构如上。每个结构体包括一个指向数组的指针，指向部分和的指针，计算的起止位置。

```

] void* sum(void* arg) {
    thread_arr *data = (thread_arr*)arg;
    int sum = 0;
]   for (int i = data->start; i < data->end; i++) {
        sum += data->arr[i];
    }
    *(data->part_sum) = sum;
- }
}

```

线程创建和合并的方法与矩阵乘法相同。

传入函数如上图，分别对每个部分的数组求和。

//合并线程并求和

```

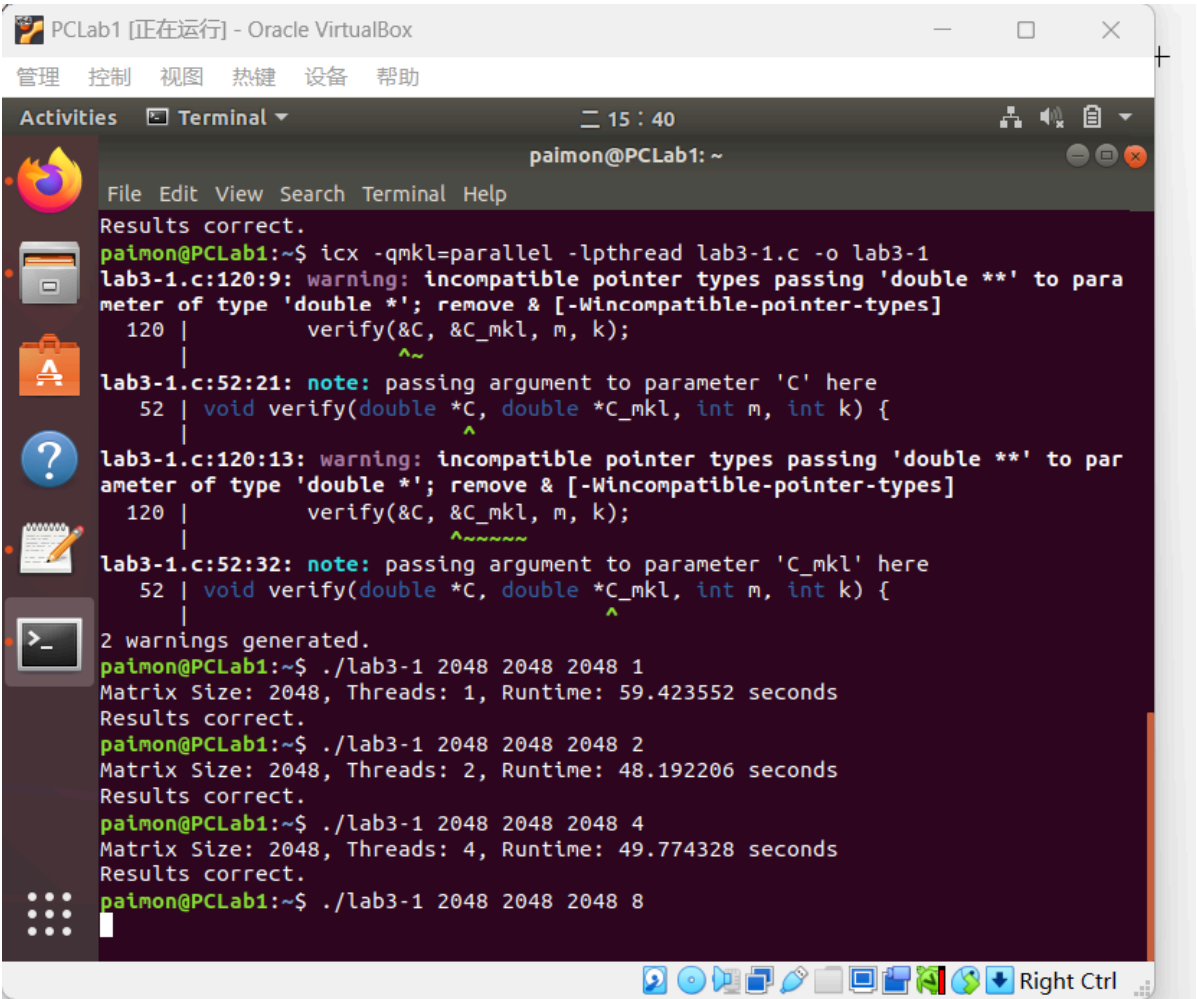
for (int i = 0; i < threads; i++) {
    pthread_join(thread[i], NULL);
}
int arr_sum = 0;
for (int i = 0; i < threads; i++) {
    arr_sum += part_sums[i];
}

```

合并线程后，将部分和加起来，形成总和。

数组求和的验证很简单。在初始化数组时顺便做加法即可求得标准值，验证运算只需要逐个比较即可。如果结果一致，输出“Calculation Correct!”, 否则输出“Wrong Answer! ”。

实验结果



矩阵乘法运算的运行截图。

进程数	矩阵规模	矩阵规模	矩阵规模	矩阵规模	矩阵规模
/	128	256	512	1024	2048
1	0.002022 正确	0.008194 正确	0.488275 正确	4.715773 正确	59.423552 正确
2	0.002484 正确	0.008622 正确	0.496233 正确	4.200340 正确	48.192206 正确
4	0.003470 正确	0.009577 正确	0.502479 正确	4.387300 正确	49.774328 正确
8	0.005317 正确	0.014196 正确	0.476258 正确	4.781588 正确	48.308156 正确
16	0.008059 正确	0.016732 正确	0.482177 正确	4.559945 正确	47.873530 正确

矩阵乘法运算的运行时间统计。可以看出，在矩阵规模较大时，采用并行可以很好地缩短运行时间，提高计算效率、提升可扩展性。在矩阵规模较小时，由于创建与合并线程的开销，并行对性能的提升不明显。

```

time: 0.099159 seconds
paimon@PCLab1:~$ gcc -o lab3-2 lab3-2.c -lpthread
paimon@PCLab1:~$ ./lab3-2 64000000 1
Calculation Correct!Array Size: 64000000, Threads: 1, Answer: 1901630021, Runtime: 0.200939 seconds
paimon@PCLab1:~$ gcc -o lab3-2 lab3-2.c -lpthread
paimon@PCLab1:~$ ./lab3-2 64000000 2
Calculation Correct!Array Size: 64000000, Threads: 2, Answer: 1901630021, Runtime: 0.183963 seconds
paimon@PCLab1:~$ gcc -o lab3-2 lab3-2.c -lpthread
paimon@PCLab1:~$ ./lab3-2 64000000 4
Calculation Correct!Array Size: 64000000, Threads: 4, Answer: 1901630021, Runtime: 0.190514 seconds
paimon@PCLab1:~$ gcc -o lab3-2 lab3-2.c -lpthread
paimon@PCLab1:~$ ./lab3-2 64000000 8
Calculation Correct!Array Size: 64000000, Threads: 8, Answer: 1901630021, Runtime: 0.188655 seconds
paimon@PCLab1:~$ gcc -o lab3-2 lab3-2.c -lpthread
paimon@PCLab1:~$ ./lab3-2 64000000 16
Calculation Correct!Array Size: 64000000, Threads: 16, Answer: 1901630021, Runtime: 0.209245 seconds
paimon@PCLab1:~$ gcc -o lab3-2 lab3-2.c -lpthread

```

数组求和的运行截图。

进程数	数组规模	数组规模	数组规模	数组规模	数组规模
/	1M	4M	32M	64M	128M
1	0.003580 正确	0.016022 正确	0.104610 正确	0.200939 正确	0.391642 正确
2	0.002484 正确	0.013847 正确	0.092924 正确	0.183963 正确	0.364636 正确
4	0.004670 正确	0.012002 正确	0.101689 正确	0.190514 正确	0.385170 正确
8	0.004741 正确	0.015119 正确	0.100128 正确	0.188655 正确	0.377470 正确
16	0.006591 正确	0.014676 正确	0.099159 正确	0.209245 正确	0.209245 正确

数组求和中，并行对性能的提升不明显。可能是因为求和运算比较简单，并行节省的时间和创建线程合并线程的花费时间相互抵消，总体运算时间、运算效率、可扩展性提升不明显。

实验感想

在本次实验中，我学会了使用 `pthread` API 创建与合并线程，从而实现并行计算。在本次实验中，我遇到的最大问题依然是内存管理问题。不过相比之前的 MPI，Pthread 并行的内存管理相对简单，我就意识到是没有为线程动态分配内存的原因，顺利解决了这个问题。此外，由于忘记了如何运行 MKL 矩阵乘法，我还花费了一点时间翻看之前的实验报告。